Increasing Throughput with Concurrency Control for ANN Indexes

Collin Drake
University of Colorado Boulder
collin.drake@colorado.edu

Abstract

Implementing thread-safety in ANN indexes presents a variety of design choices. Ultimately, the tradeoffs converge to two axes: locking granularity and conflict resolution. In both architectures, conflicts may be handled optimistically or pessimistically. In this paper, we implement and evaluate all four combinations of these two axes against industry-standard implementations.

1. Introduction

As vector databases increasingly support streaming workloads, the need for concurrent queries against continuously updated indexes has become apparent [1]. While approximate nearest neighbor (ANN) search has been extensively studied under the assumption of a static index [2], the concurrency control mechanisms required for dynamic workloads remain underexplored [3]. Existing systems either sacrifice read-write concurrency entirely or rely on batch processing strategies incompatible with streaming insertions.

Existing approaches to parallel ANN index construction fall short of these requirements. FAISS, a well-known example of fine-grained locking, utilizes this locking strictly for parallel index construction (protecting concurrent writers), rather than for isolation between readers and writers [4]. Consequently, it does not support simultaneous search and update. Furthermore, while FAISS exploits thread-level parallelism to batch-process search queries, the search path itself is completely lock-free [5].

ParlayANN takes a different approach, offering two strategies for parallel construction. Its deterministic batch construction uses prefix sums and parallel primitives to process points in a fixed order, ensuring reproducible results across runs. For incremental updates, it employs lock-free batch insertion using compare-and-swap operations, allowing threads to modify neighbor lists without acquiring locks [6]. However, neither technique is well-suited for online systems. Prefix sum-based construction requires the entire batch of points to be known upfront; offsets must be computed before any insertions can proceed, making it fundamentally incompatible with a stream of arriving vectors. Similarly, batch conflict scheduling assumes all operations are available for analysis before execution, enabling the scheduler to order them to minimize conflicts. In an online setting, there is no batch to analyze.

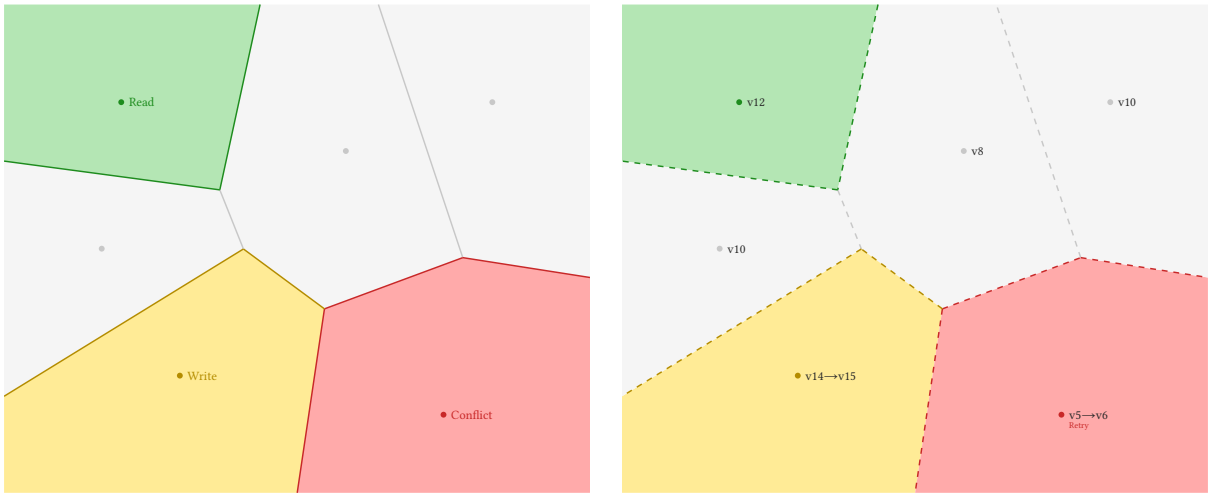
JVector demonstrates near-linear scaling of nonblocking concurrent construction on SIFT-1M [7], but like FAISS and ParlayANN this targets build-time parallelism rather than online read-write concurrency.

We address this gap directly. Concurrency control for ANN indexes varies along two axes: locking granularity (coarse-grained per partition or bucket vs. fine-grained per neighbor list) and conflict resolution (pessimistic blocking vs. optimistic validation and retry) [8]. We implement all four combinations for both IVFFlat and HNSW and benchmark them under mixed read-write workloads.

2. Methodology

2.1. Index Implementations

Each index family (IVFFlat and HNSW) is implemented with four concurrency strategies spanning two axes: locking granularity (coarse vs. fine) and conflict resolution (pessimistic vs. optimistic). All variants use SIMD-accelerated distance computation via a common module. All IVFFlat variants additionally apply four-element prefetching in the scan inner loop.



Pessimistic

```
T1: search → acquire green (shared)
T2: insert → acquire red (exclusive)
T3: insert → acquire yellow (exclusive)
T4: insert → acquire red (blocked)
T1: scan green → release
T2: write red → release → T4 unblocks
T3: write yellow → release
T4: write red → release
```

Optimistic

```
// red: v=5; green: v=12; yellow: v=14
T1: search → snapshot red v=5
T2: insert → write to red v5→v6
T4: insert → write to yellow v14→v15
T3: search → snapshot green v=12
T1: validate red: 5 ≢ 6 → retry
T3: validate green: 12 = 12
```

Figure 1: Geometric interpretation of IVFFlat coarse locking strategies over a Voronoi partition. *Pessimistic*: each cell is guarded by a mutex; readers acquire a shared lock and writers an exclusive lock. *Optimistic*: threads snapshot the cell version before operating and validate on commit; a writer that advances the version mid-search forces the reader to retry.

2.1.1. HNSW-IVF Hybrid

The four concurrency strategies above apply uniformly within each index family, but the structural asymmetry between HNSW and IVFFlat suggests a hybrid that inherits the strengths of both. HNSW's upper layers naturally partition the vector space: each node promoted to layer 2 or above implicitly defines a Voronoi cell over the lower-layer nodes nearest to it. These upper-layer nodes are the hub nodes of the graph – a small, well-connected subset responsible for most long-range traversal [9] – making them effective spatial landmarks for partitioning. By making this partition explicit, we obtain IVF-style write isolation at the base layer while retaining HNSW's sublinear graph walk for search.

The choice of layer 2 rather than layer 1 as the partition threshold follows from the geometry of HNSW's level distribution. Each HNSW layer contains roughly $1/M$ of the nodes from the layer below. For N indexed vectors, layer 1 contains approximately N/M nodes and layer 2 contains approximately N/M^2 nodes. Using layer-1 nodes as partition centers produces N/M partitions with only $\sim M$ vectors per cell on average. At this granularity the partitions are too sparse for productive IVF-style probing: reaching acceptable recall requires probing many nearly-empty cells, and the per-cell overhead dominates. Layer 2, by contrast, yields N/M^2 partitions with $\sim M^2$ vectors per cell, which for typical values of M falls near the standard IVF heuristic of $\sqrt{N}$ clusters. For example, with $M = 16$ and $N = 1\,000\,000$, layer 1 contains roughly 31,000 nodes ($\sim$32 vectors per cell) while layer 2 contains roughly 1,000 nodes ($\sim$1,000 vectors per cell).

Our `HybridPessimistic` implementation maintains a full HNSW graph across all layers, including layer 0. Each layer-2+ node additionally registers a partition upon insertion, and each lower-layer node is assigned to the partition whose center was nearest during the inserting thread's descent. The partition assignment determines which lock protects a node's layer-0 edge list; it does not restrict the search path. Layer-0 edges cross partition boundaries freely, connecting each node to its M nearest neighbors regardless of partition membership.

The first nodes that arrive before any layer-2+ node appears are held in an unassigned queue and drained into their nearest center when one appears, using brute-force nearest-center lookup over this small set. Deletion of a partition center triggers reassignment: the center's children are redistributed to their nearest surviving center, and the center's upper-layer graph edges are severed. All deleted nodes are tombstoned and filtered during graph traversal.

Search proceeds in two phases. First, greedy descent through the upper HNSW layers routes the query to layer 2, where a beam search identifies the `nprobe` nearest partition centers. Second, a layer-0 beam search is launched from each probed center, walking edges across partition boundaries. As each node is encountered, its partition's shared lock is acquired to read the edge list. The `nprobe` parameter controls how many entry points seed the layer-0 walk: additional entry points explore spatially distinct regions of the graph, improving recall for queries near partition boundaries.

The locking scheme separates the concurrency domain from the search domain. Upper-layer graph edges are protected by per-layer `shared_mutex` locks (as in `HNSWCoarsePessimistic`). At layer 0, each partition has its own `shared_mutex` protecting the edge lists of all nodes assigned to it. Readers acquire shared partition locks as they traverse nodes, so multiple concurrent searches never block each other. Writers acquire exclusive locks on the partitions they modify: the new node's own partition for its edge list, plus each neighbor's partition for reverse edges, acquired in index order to prevent deadlock. Because a typical insert touches at most 2-3 partitions (bounded by M), writers to disjoint regions proceed without contention: the same concurrency property that makes IVFFlat scale well, but with sublinear search rather than linear flat scan.

The tradeoff is that partition quality depends on HNSW's stochastic level generation rather than k-means training. The resulting partitions are less balanced than IVFFlat's, and partition sizes follow the distribution of the data rather than an optimized quantizer. However, because the layer-0 search walks edges across partition boundaries rather than scanning within them, recall is less sensitive to partition quality than in pure IVF. Even `nprobe` $= 1$ can achieve high recall if the graph is well-connected. The routing cost through the upper layers is logarithmic in the number of partitions rather than linear, since HNSW's graph replaces the brute-force centroid scan.

2.2. Workload

The benchmarking harness is implemented in Python and invokes the core C++ index implementations through bindings to minimize orchestration overhead.

The primary workload consists of a concurrent mix of document insertions and approximate nearest neighbor searches. We vary the number of concurrent worker threads across seven points – 2, 4, 8, 12, 16, 20, and 24 – spanning the full physical core count of the test machine and capturing behavior through the E-core saturation region into combined P+E load.

We evaluate each index configuration under two write-ratio bands, both ramping linearly across the thread-count sweep:

- **Production band (W: 1%–5%)**. The write ratio increases from 1% at 2 threads to 5% at 24 threads. Standard database benchmarks model production workloads as overwhelmingly read-heavy. YCSB Workload B (95% read, 5% update) and Workload D (95% read, 5% insert) represent the read-mostly tier of the Yahoo! Cloud Serving Benchmark suite [10], and even the "update-heavy" Workload A allocates only 50% of operations to writes. Production vector databases skew further toward reads: ingestion is typically batched or periodic, while queries arrive continuously. Our 1–5% range falls within this production regime.
- **Stress-test band (W: 20%–50%)**. The write ratio increases from 20% at 2 threads to 50% at 24 threads. At these ratios, write-write and read-write conflicts become frequent enough to expose behavioral differences between concurrency control strategies that the production band masks. This band is not intended to represent a deployment profile; it isolates the locking and conflict-resolution overhead that is the focus of this paper.

We evaluate our implementations using the SIFT-128 ($n = 1\,000\,000$), Fashion-MNIST-784 ($n = 60\,000$) [11], GloVe ($n = 1\,183\,514$), GIST-960 ($n = 1\,000\,000$), NYTimes-256 ($n = 290\,000$), and MNIST-784 ($n = 60\,000$) datasets [12].

2.3. Measurement

Each throughput data point proceeds in two phases. First, a *construction phase* creates a fresh index, trains it (for IVF variants), and bulk-inserts the first 50% of the dataset into the shared index, single-threaded. This pre-load gives the index enough structure to serve queries before the timed phase begins. Build time covers this phase and is reported separately; it is excluded from the throughput calculation. Second, a *mixed-workload phase* launches writer and reader threads concurrently against the pre-loaded index: writers insert the remaining vectors (split evenly across insert threads) while readers execute five full passes over the query set. All performance-critical C++ methods release the Python GIL, so threads achieve true parallelism. When a C++ call returns, however, the thread must reacquire the GIL before Python can dispatch the next operation; if another thread holds the GIL at that instant, the thread stalls despite having completed its index work. This queuing at the GIL boundary is absorbed into the wall-clock window, making absolute throughput figures a conservative lower bound on what the underlying indexes can sustain. Wall-clock time is retained rather than instrumented C++ time to maintain consistency with external library baselines (FAISS, USearch, Weaviate, Redis), which cannot be measured otherwise; since all implementations share the same harness overhead, relative comparisons remain valid. Throughput is computed as total operations (inserts plus searches) divided by the wall-clock duration of this second phase only.

All indexes return $k = 10$ nearest neighbors per query. HNSW variants use $M = 16$ bidirectional links per node and $\mathrm{ef}_{\mathrm{construction}} = 200$. IVFFlat variants use $n_{\mathrm{list}} = \left\lfloor \sqrt{N} \right\rfloor$ clusters, where N is the number of indexed vectors, and $n_{\mathrm{probe}} = 1$ for throughput benchmarks.

All experiments were conducted on a Lenovo Legion Pro 7i 16IAX10H with an Intel Core Ultra 9 275HX CPU (24 total cores: 8P+16E) and 32 GiB RAM. Our evaluation is restricted to CPU-based architectures to maintain a direct comparison between our custom concurrency control strategies. Although GPU-acceleration is a common optimization for batch ANN search, implementing our proposed fine-grained locking and optimistic retry mechanisms within GPU kernels involves significant architectural complexity that we reserve for future work.

3. Results

3.1. Throughput

Incidentally, the lower a dataset's throughput, the higher is cluster density.

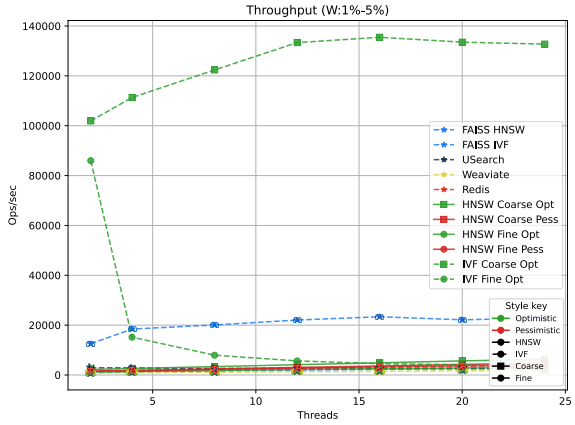


Figure 3: SIFT-128

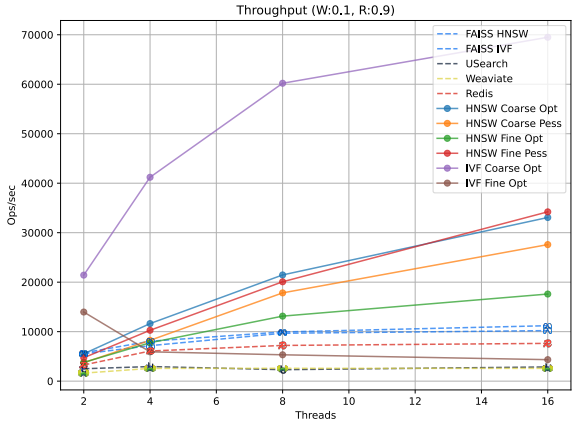


Figure 4: Fashion-MNIST-784

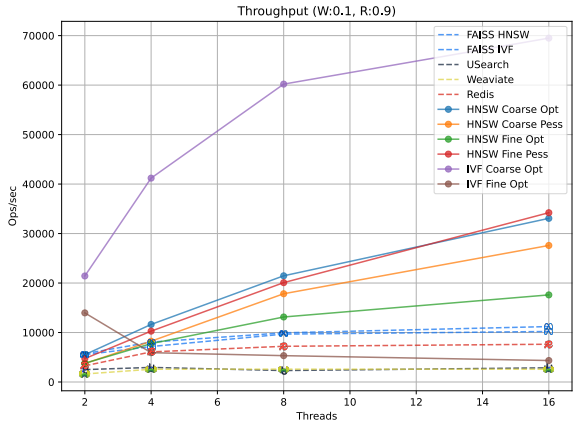


Figure 5: Fashion-MNIST-784

Figure 2: Throughput scaling with increasing numbers of threads for each index type, for tested full-run datasets.

4. Discussion

- ***Why hasn't this been done before?*** The predominant paradigm has been offline construction followed by read-only serving, so reader-writer concurrency at the index level simply never arose. Systems that do accept writes typically side-step the problem architecturally: mutable segments are flushed to read-only storage on a rolling basis, and searches hit only sealed data [13]. Deletions are handled similarly via tombstoning followed by periodic rebuilds rather than in-place graph repair [14]. Streaming workloads driven by retrieval-augmented generation [15] have only recently made in-place concurrent update a practical consideration.

- ***Why does HNSW degrade more under concurrent writes than IVF?*** The answer lies in a structural asymmetry between the two index types. HNSW is a navigable small-world graph: its search correctness depends on the graph remaining well-connected across layers [16]. During insertion, a new node is wired into the graph by selecting neighbors via a heuristic and then back-linking those neighbors to the new node [16]. Under concurrent writes, these two steps are not atomic. A racing writer can observe a partially-linked node, traverse a stale edge, or have its own neighbor list pruned before its back-links are established, any of which can leave the graph with weakly connected or entirely isolated nodes. Heuristics like deferred or batched pruning reduce the frequency of such breaks but cannot eliminate them: they trade recall loss for throughput, rather than recovering the full structural guarantee. The damage is disproportionate when hub nodes are affected, since a small hub subset carries most long-range traversal [9]. IVF does not share this vulnerability. At the coarse quantizer level, cluster assignment is a read-only operation; inserting a vector into a cluster appends to a list and requires only a per-cluster lock [4]. Concurrent writers targeting different clusters are entirely independent, and even writers within the same cluster contend only on a flat append structure with no graph invariant to preserve. The degradation seen in HNSW throughput and recall under high write concurrency is therefore not merely an implementation artifact; it reflects a fundamental tension between the graph connectivity invariant that makes HNSW fast and the atomicity that concurrent mutation requires.

5. Conclusion

Bibliography

[1] S. Gong, H. Sun, Z. Fang, and L. Liu, "VStream: A Distributed Streaming Vector Search System," *Proceedings of the VLDB Endowment*, vol. 18, no. 6, 2025, doi: 10.14778/3725688.3725692.

[2] L. Ma *et al.*, "A Comprehensive Survey on Vector Database: Storage and Retrieval Technique, Challenge." [Online]. Available: https://arxiv.org/abs/2310.11703

[3] A. Singh, S. J. Subramanya, R. Krishnaswamy, and H. V. Simhadri, "FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search." [Online]. Available: https://arxiv.org/abs/2105.09613

[4] M. Douze *et al.*, "The Faiss Library." [Online]. Available: https://arxiv.org/abs/2401.08281

[5] Meta Platforms, Inc., "FAISS: IndexHNSW Implementation." [Online]. Available: https://github.com/facebookresearch/faiss/blob/main/faiss/IndexHNSW.cpp

[6] M. D. Manohar *et al.*, "ParlayANN: Scalable and Deterministic Parallel Graph-Based Approximate Nearest Neighbor Search Algorithms." [Online]. Available: https://arxiv.org/abs/2305.04359

[7] DataStax, "JVector: The Most Advanced Embedded Vector Search Engine." [Online]. Available: https://github.com/datastax/jvector

[8] H. T. Kung and J. T. Robinson, "On Optimistic Methods for Concurrency Control," *ACM Transactions on Database Systems*, vol. 6, no. 2, pp. 213–226, 1981, doi: 10.1145/319566.319567.

[9] B. Munyampirwa, V. Lakshman, and B. Coleman, "Down with the Hierarchy: The 'H' in HNSW Stands for ``Hubs'','' in *Proceedings of the 42nd International Conference on Machine Learning*, 2025. [Online]. Available: https://arxiv.org/abs/2412.01940

[10] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking Cloud Serving Systems with YCSB," in *Proceedings of the 1st ACM Symposium on Cloud Computing*, in SoCC '10. ACM, 2010, pp. 143–154. doi: 10.1145/1807128.1807152.

[11] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms," 2017, [Online]. Available: https://arxiv.org/abs/1708.07747

[12] M. Aumüller, E. Bernhardsson, and A. Faithfull, "ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms," *Information Systems*, vol. 87, 2020, doi: 10.1016/j.is.2019.02.006.

[13] J. Wang *et al.*, "Milvus: A Purpose-Built Vector Data Management System," in *Proceedings of the 2021 ACM SIGMOD International Conference on Management of Data*, 2021, pp. 2614–2627. doi: 10.1145/3448016.3457550.

[14] A. Sage, "Great Algorithms Are Not Enough." [Online]. Available: https://www.pinecone.io/blog/hnsw-not-enough/

[15] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, 2020, pp. 9459–9474. [Online]. Available: https://arxiv.org/abs/2005.11401

[16] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020, doi: 10.1109/TPAMI.2018.2889473.